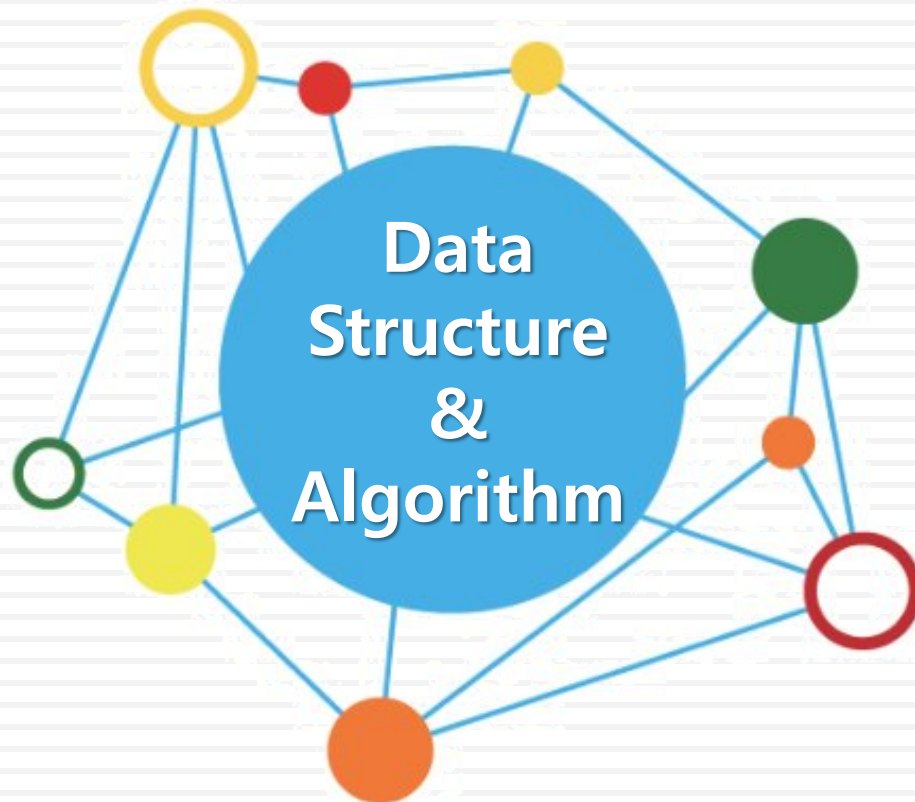


자료구조와 알고리즘



유길현
트리(Tree)

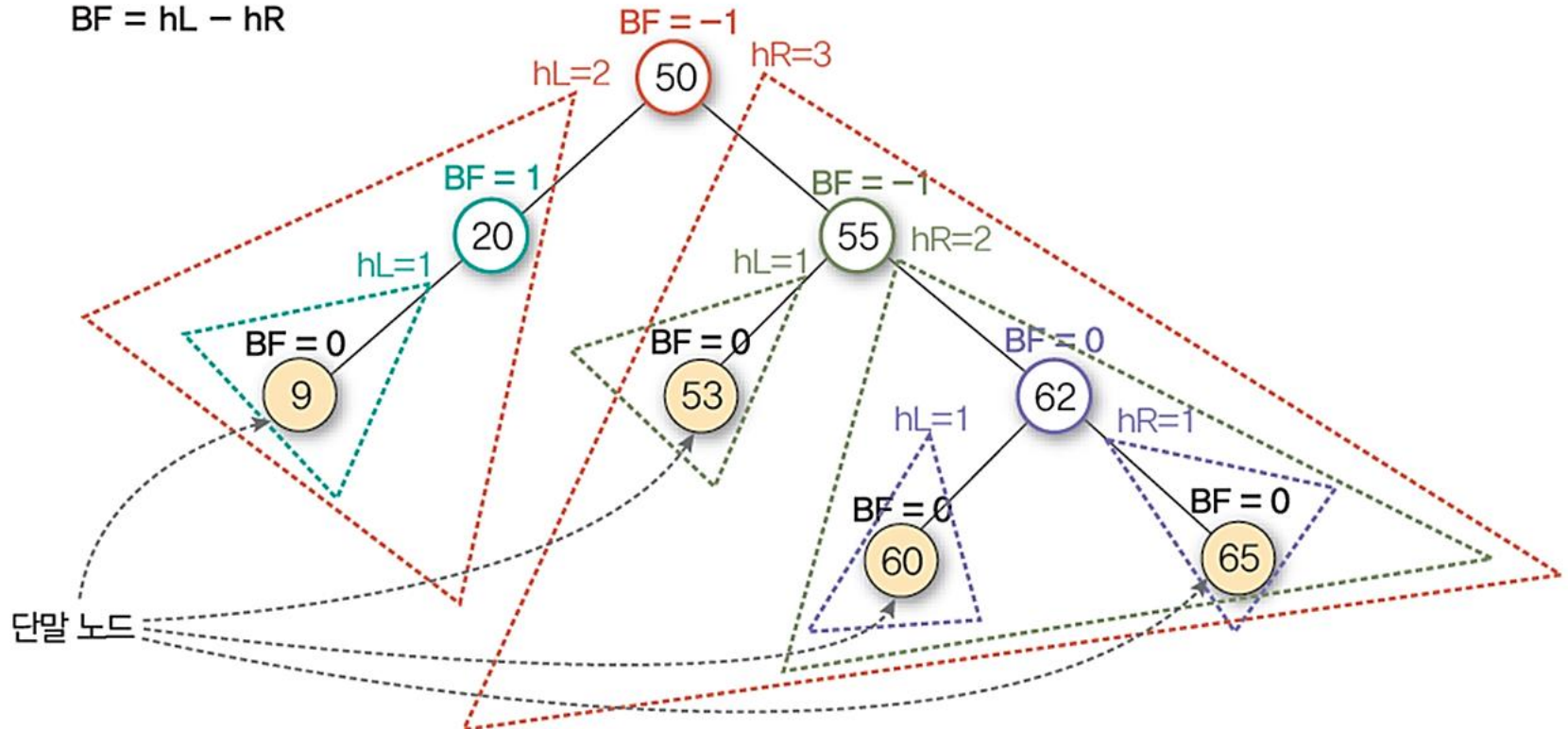
❖ AVL(Adelson-Velskii, Landis Tree) 트리의 개념과 유형

- 각 노드에서 왼쪽 서브 트리의 높이 h_L (height of Left subtree)과 오른쪽 서브 트리의 높이 h_R (height of Right subtree)의 차이가 1 이하인 트리
- 특징
 - 왼쪽 서브 트리 < 부모 노드 < 오른쪽 서브 트리의 크기 관계를 가짐
 - 각 노드의 왼쪽 서브 트리 높이와 오른쪽 서브 트리 높이의 차이($h_L - h_R$)를 노드의 균형 인수(BF, Balance Factor)라 함
 - 각 노드의 균형 인수로 $\{-1, 0, +1\}$ 값만 가지게 함으로써 왼쪽 서브 트리와 오른쪽 서브 트리의 균형을 항상 유지함

❖ AVL(Adelson-Velskii, Landis Tree) 트리의 개념과 유형

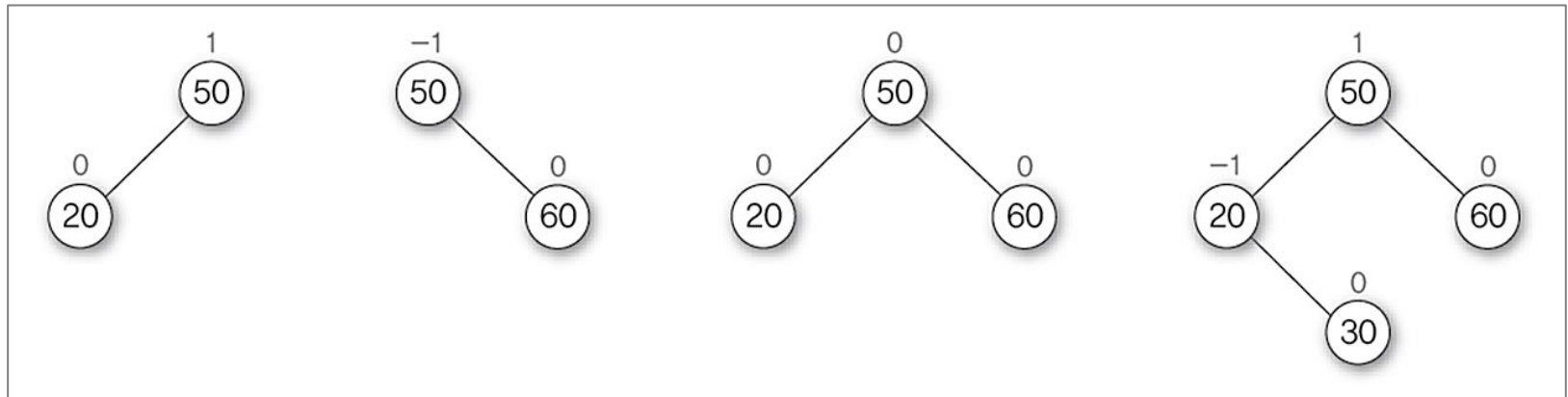
- 균형 인수 BF는 각 노드에서 왼쪽 서브 트리의 높이 hL 과 오른쪽 서브 트리의 높이 hR 의 차이 만족한다.
- AVL 트리의 균형 인수 구하기

$$BF = hL - hR$$



❖ AVL(Adelson-Velskii Landis Tree) 트리의 개념과 유형

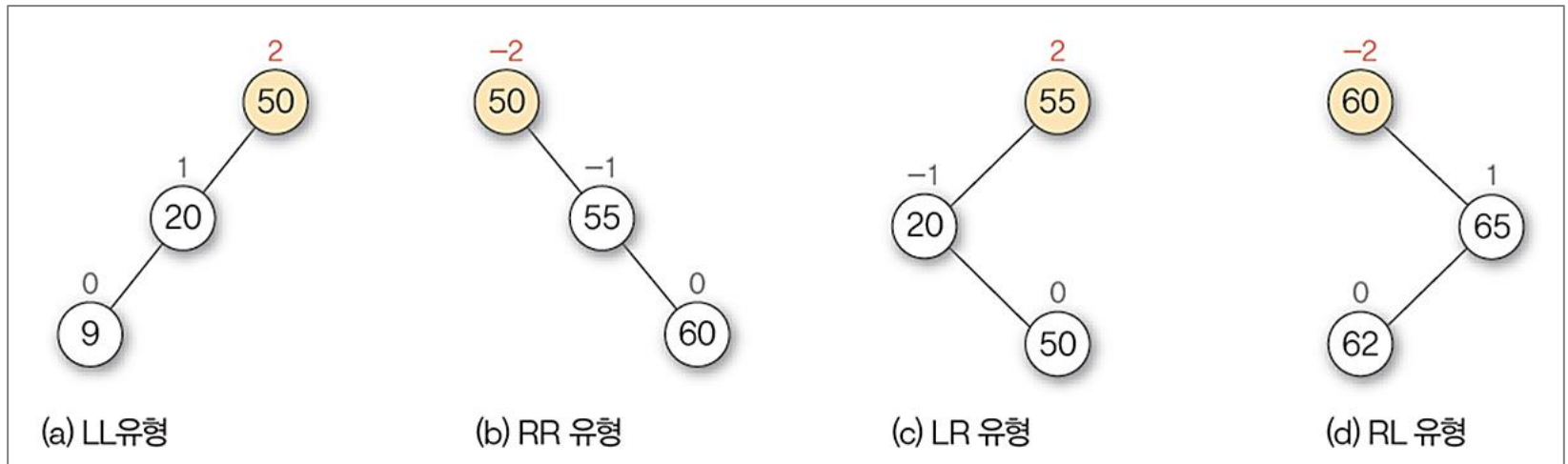
■ AVL 트리의 예



모든 노드가 균형 인수로 $\{-1, 0, 1\}$ 값을 가지므로 AVL 트리임

❖ AVL(Adelson-Velskii Landis Tree) 트리의 개념과 유형

■ 비AVL 트리의 예



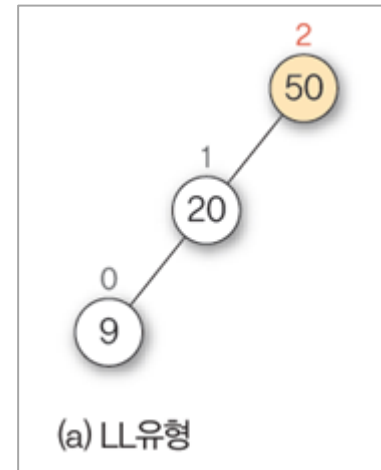
노드들이 균형 인수로 $\{-1, 0, 1\}$ 외의 값을 가지므로 균형이 깨져
한 방향으로 치우친 비AVL 트리임

(균형이 깨진 노드의 균형 인수가 $+$ 이면 왼쪽 서브 트리에 문제가
있는 것이고, 균형 인수가 $-$ 이면 오른쪽 서브 트리에 문제가 있음)

❖ AVL 트리에서 발생하는 불균형

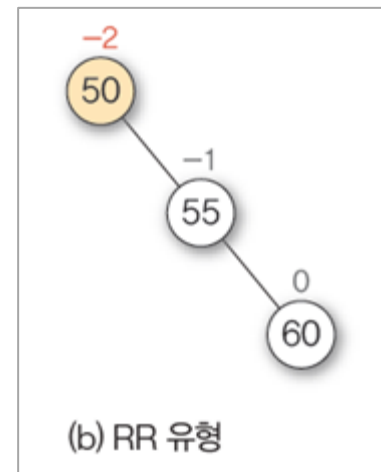
■ LL 유형 (Left-Left 유형)

(a)는 노드 50의 균형 인수가 +2로 균형이 깨진 노드인데 균형 인수가 +이므로 왼쪽서브 트리가 문제. 그리고 노드 50의 왼쪽(L) 자식인 노드 20의 균형 인수가 +이므로 노드 20의 왼쪽(L) 자식 노드가 불균형과 관련 있음. 따라서 노드 50이 불균형 노드가 된 이유는 노드 50이 왼쪽(L) 자식 노드와 자식의 왼쪽(L) 자식 노드를 가짐으로써 왼쪽으로 치우쳤기 때문임.



■ RR 유형 (Right-Right 유형)

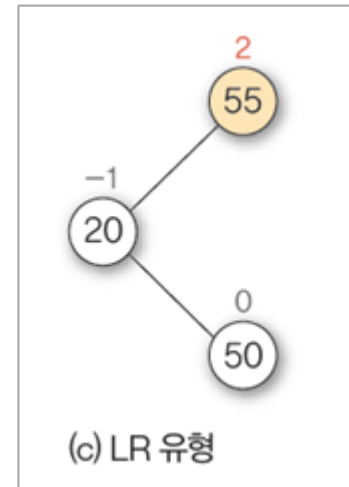
(b)는 노드 50의 균형 인수가 -2로 균형이 깨진 노드인데 균형 인수가 -이므로 오른쪽 서브 트리에 문제임. 그리고 노드 50의 오른쪽(R) 자식인 노드 55의 균형 인수가 -이므로 노드 55의 오른쪽(R) 자식 노드가 불균형과 관련 있음. 따라서 노드 50이 불균형 노드가 된 이유는 노드 50이 오른쪽(R) 자식 노드와 자식의 오른쪽(R) 자식 노드를 가짐으로써 오른쪽으로 치우쳤기 때문임



❖ AVL 트리에서 발생하는 불균형

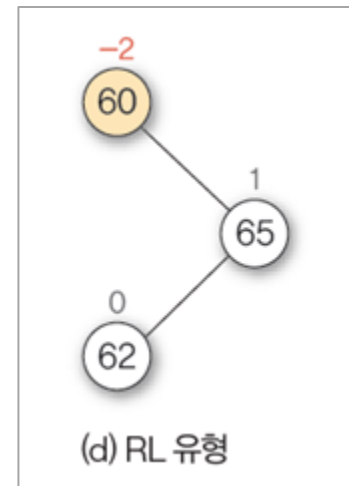
■ LR 유형 (Left-Right 유형)

(c)는 노드 55의 균형 인수가 +2로 균형이 깨진 노드인데 균형 인수가 +이므로 왼쪽 서브 트리에 문제임. 그리고 노드 55의 왼쪽(L) 자식인 노드 20의 균형 인수는 -이므로 노드 20의 오른쪽(R) 자식 노드가 불균형과 관련 있음. 따라서 노드 55가 불균형 노드가 된 이유는 왼쪽(L) 자식 노드와 자식의 오른쪽(R) 자식 노드로 인해 왼쪽 서브 트리가 치우쳤기 때문임



■ RL 유형 (Right-Left 유형)

(d)는 노드 60의 균형 인수가 -2로 균형이 깨진 노드인데 균형 인수가 -이므로 오른쪽 서브 트리에 문제임. 그리고 노드 60의 오른쪽(R) 자식인 노드 65의 균형 인수는 +이므로 노드 65의 왼쪽(L) 자식 노드가 불균형과 관련 있음. 따라서 노드 60이 불균형 노드가 된 이유는 오른쪽(R) 자식 노드와 자식의 왼쪽(L) 자식 노드로 인해 오른쪽 서브 트리가 치우쳤기 때문임



❖ AVL 트리의 회전 연산

- AVL 트리에서 수행하는 삽입·삭제 작업은 이진 탐색 트리에서의 삽입·삭제 작업과 같고, 이후에 균형을 맞추어주는 재구성 작업이 추가되는데 이 작업은 회전Rotation 연산을 통해 이루어짐
 - 단순 회전Single Rotation : LL 회전과 RR 회전과 같이 한 번 회전하는 것
 - 이중 회전Double Rotation : LR 회전과 RL 회전과 같이 두 번 회전하는 것

불균형의 유형과 원인	해결방법
LL유형 : 불균형 발생 노드의 왼쪽 자식 노드와 자식의 왼쪽 자식 노드에 의해 왼쪽으로 치우침	LL 회전 : 문제 구간 중 상위 구간을 오른쪽으로 회전시킴
RR 유형 : 불균형 발생 노드의 오른쪽 자식 노드와 자식의 오른쪽 자식 노드에 의해 오른쪽으로 치우침	RR 회전 : 문제 구간 중 상위 구간을 왼쪽으로 회전시킴
LR 유형 : 불균형 발생 노드의 왼쪽 자식 노드와 자식의 오른쪽 자식 노드에 의해 왼쪽 서브 트리가 치우침	LR 회전 : 문제 구간 중 하위 구간을 왼쪽으로 1차 회전시켜 LL유형으로 변환한 다음 LL 회전을 적용함
RL 유형 : 불균형 발생 노드의 오른쪽 자식 노드와 자식의 왼쪽 자식 노드에 의해 오른쪽 서브 트리가 치우침	RL 회전 : 문제 구간 중 하위 구간을 오른쪽으로 1차 회전시켜 RR 유형으로 변환한 다음 RR 회전을 적용함

❖ LL 회전 연산

- 삽입, 삭제 연산 후에 AVL 트리에 LL 유형의 불균형이 발생했을 때 적용



알고리즘

AVL 트리의 LL회전

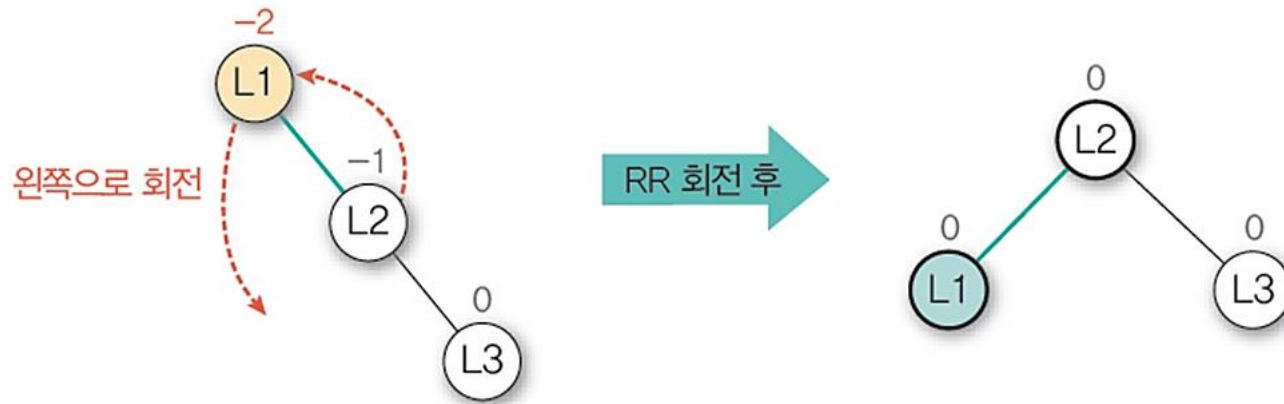
LL_rotate(L1)

- ① L2의 오른쪽 자식 노드를 L1의 왼쪽 자식 노드로 옮긴다.
- ② L1을 L2의 오른쪽 자식 노드로 옮긴다.

end LL_rotate()

❖ RR 회전 연산

- 삽입, 삭제 연산 후에 AVL 트리에 RR 유형의 불균형이 발생했을 때 적용



알고리즘

AVL 트리의 RR회전

RR_rotate(L1)

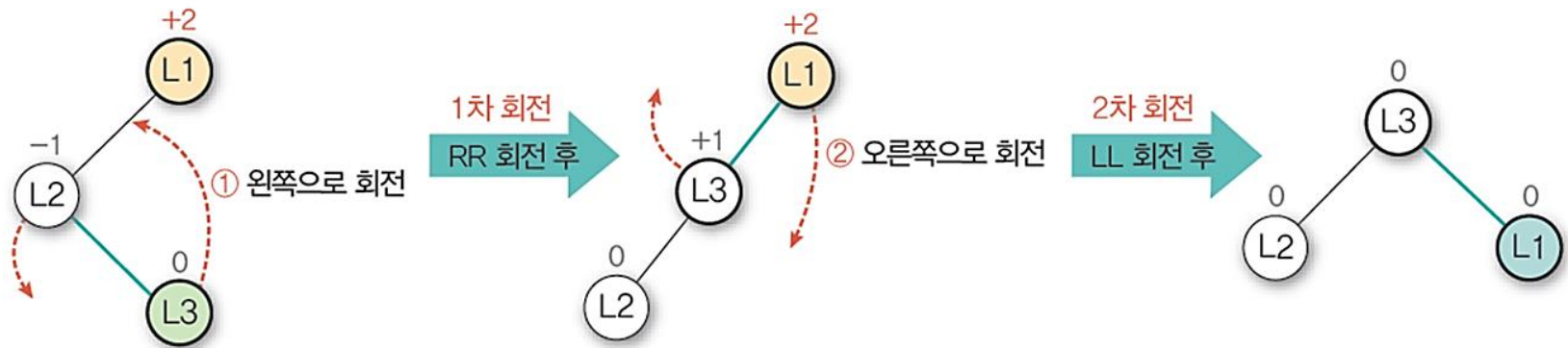
① L2의 왼쪽 자식 노드를 L1의 오른쪽 자식 노드로 옮긴다.

② L1을 L2의 왼쪽 자식 노드로 옮긴다.

end RR_rotate()

❖ LR 회전 연산

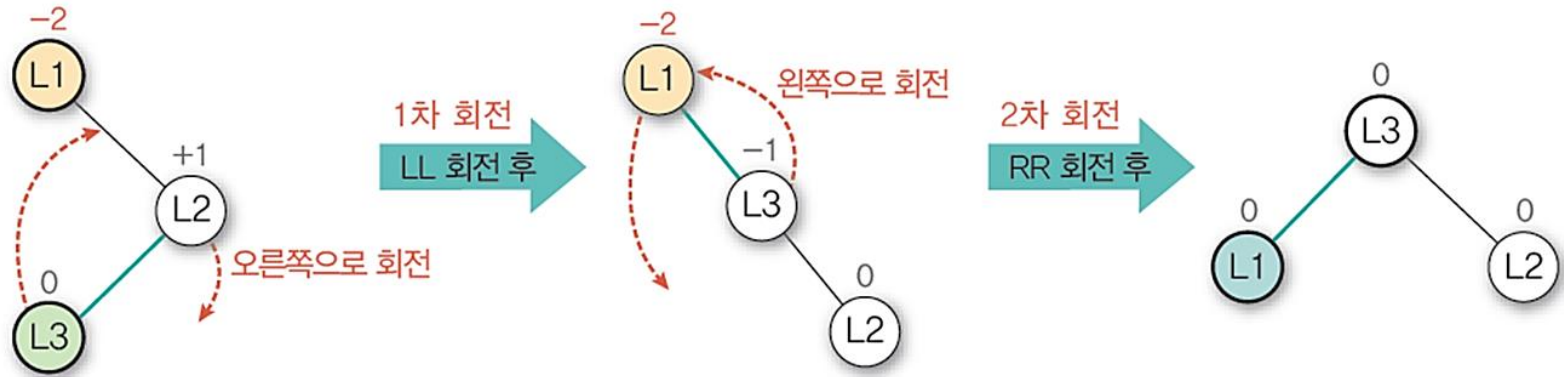
- 삽입, 삭제 연산 후에 AVL 트리에 LR 유형의 불균형이 발생했을 때 적용



알고리즘	AVL 트리의 LR회전
<pre> LR_rotate(L1) ① L2와 L3 구간에 대한 RR_rotation(L2)를 수행하고 반환된 노드 L3을 L1 의 왼쪽 자식 노드로 만든다. ② L1에 대해서 LL_rotatte(L1)을 수행한다. end LR_rotate() </pre>	

❖ RL 회전 연산

- 삽입, 삭제 연산 후에 AVL 트리에 RL 유형의 불균형이 발생했을 때 적용



알고리즘

AVL 트리의 RL회전

LR_rotate(L1)

- ① L2와 L3 구간에 대한 LL_rotation(L2)를 수행하고 반환된 노드 L3을 L1의 오른쪽 자식 노드로 만든다.
- ② L1에 대해서 RR_rotatte(L1)을 수행한다.

end RL_rotate()

❖ AVL 트리 회전 연산 예

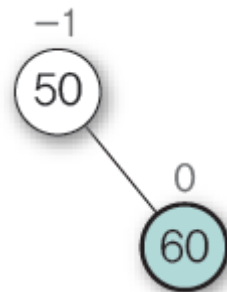
- {50, 60, 70, 90, 80, 75, 73, 72, 78} 원소 아홉 개를 AVL 트리에 삽입하면서 불균형이 발생했을 때 회전 연산을 수행하여 균형 상태를 유지하는 예

(초록색 노드는 삽입 노드, 노란색 노드는 불균형 발생한 노드. 불균형 유형에 해당하는 세 개의 노드는 초록색 간선으로 표시)

- ① 노드 50 삽입 : 공백 AVL 트리에 노드 50을 삽입



- ② 노드 60 삽입 : 노드 50 오른쪽 자식으로 노드 60을 삽입

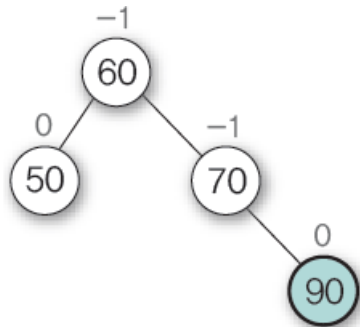


❖ AVL 트리 회전 연산 예

- ③ 노드 70 삽입 : 노드 60의 오른쪽 자식으로 노드 70을 삽입. 하지만 노드 50의 균형 인수가 -2가 되어 RR 유형의 불균형이 발생하므로 RR 회전을 수행하여 균형을 회복

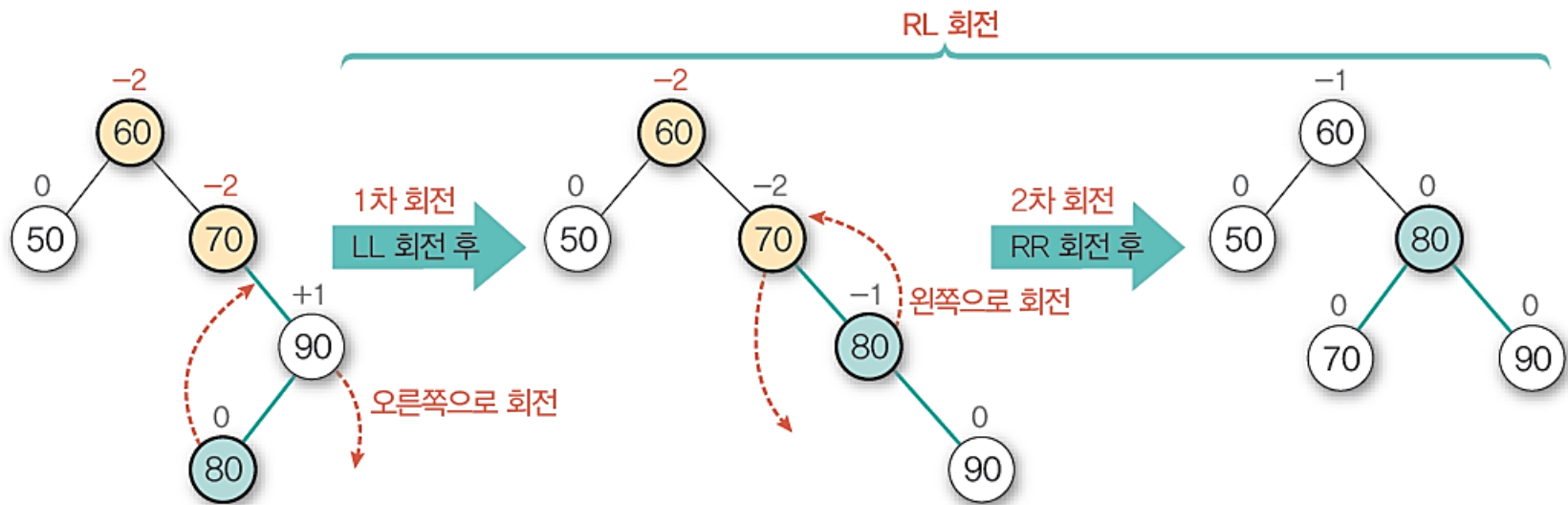


- ④ 노드 90 삽입 : 노드 70의 오른쪽 자식으로 노드 90을 삽입



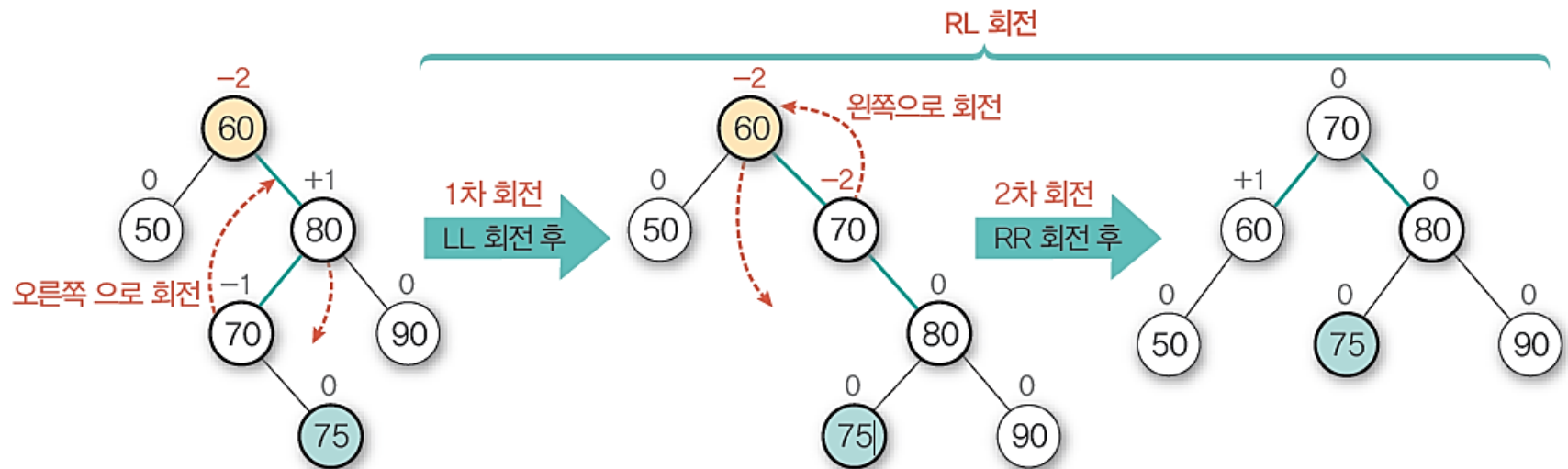
❖ AVL 트리 회전 연산 예

- ⑤ 노드 80 삽입 : 노드 90의 왼쪽 자식으로 노드 80을 삽입. 노드 70의 균형 인수가 -2가 되어 RL유형의 불균형이 발생하므로 1차 회전에서 LL 회전을 하고 2차 회전에서 RR 회전을 하는 RL 회전을 수행하여 균형을 회복



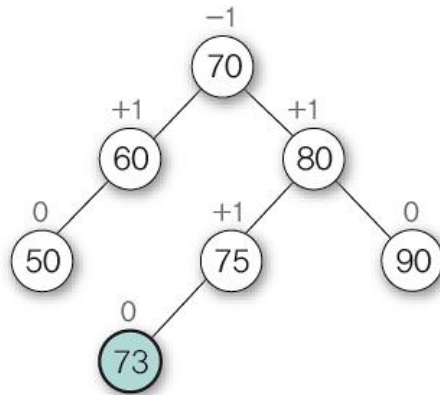
❖ AVL 트리 회전 연산 예

- ⑥ 노드 75 삽입 : 노드 70의 오른쪽 자식으로 노드 75를 삽입. 노드 60의 균형 인수가 -2가 되었고, RL 유형의 불균형이 발생하였으므로 1차 회전에서 LL 회전을 하고 2차 회전에서 RR 회전을 하는 RL회전을 수행하여 균형을 회복

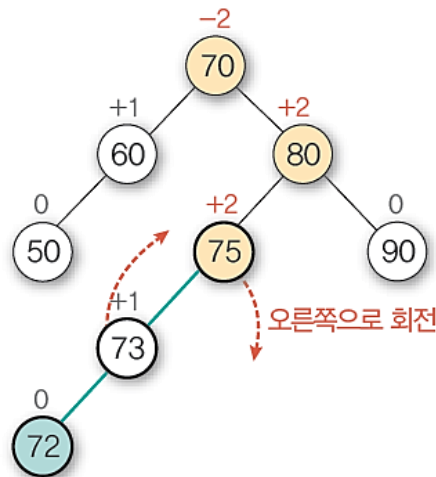


❖ AVL 트리 회전 연산 예

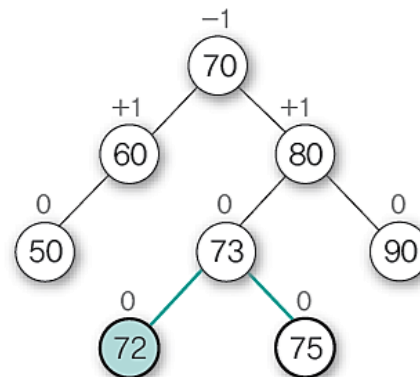
⑦ 노드 73 삽입 : 노드 75의 왼쪽 자식으로 노드 73을 삽입



⑧ 72 삽입 : 노드 75의 균형 인수가 +2가 되면서 LL 유형의 불균형이 발생 LL 회전 연산을 수행하여 균형을 회복

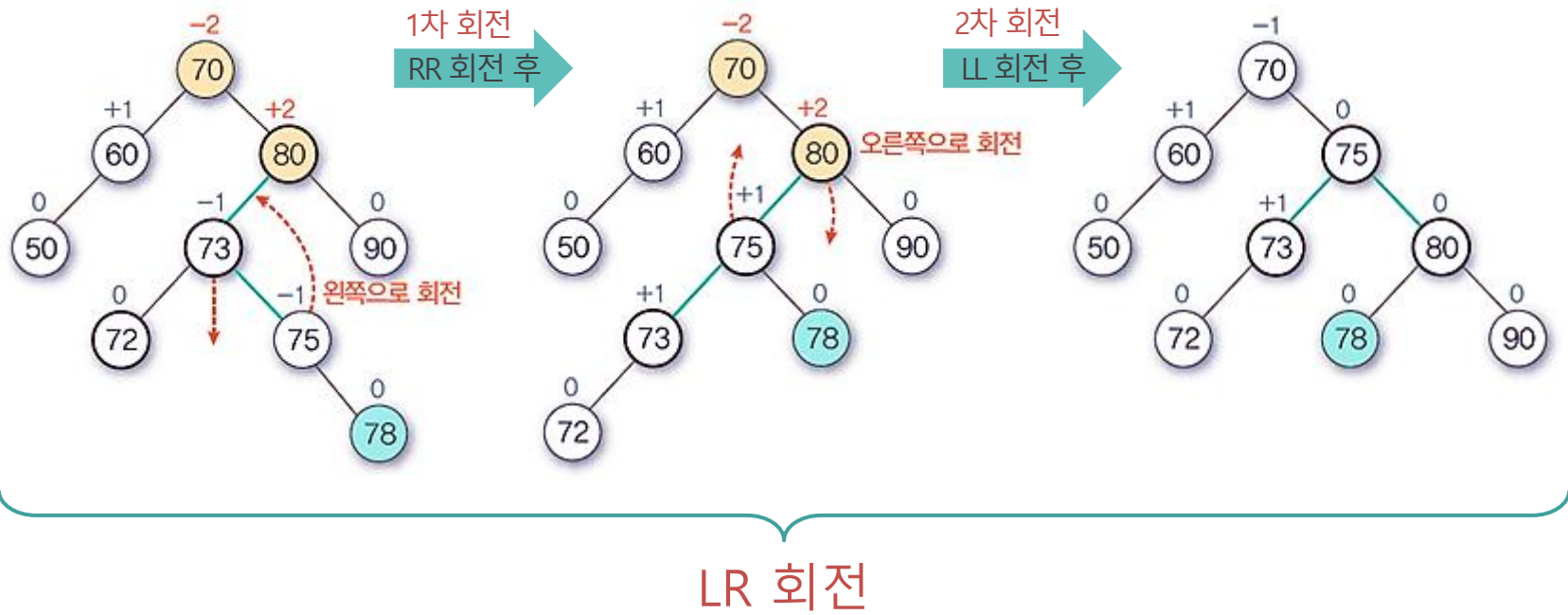


LL 회전 후

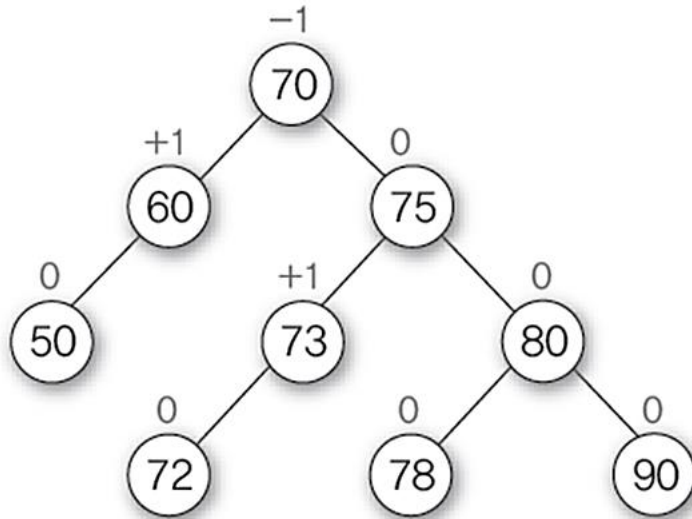


❖ AVL 트리 회전 연산 예

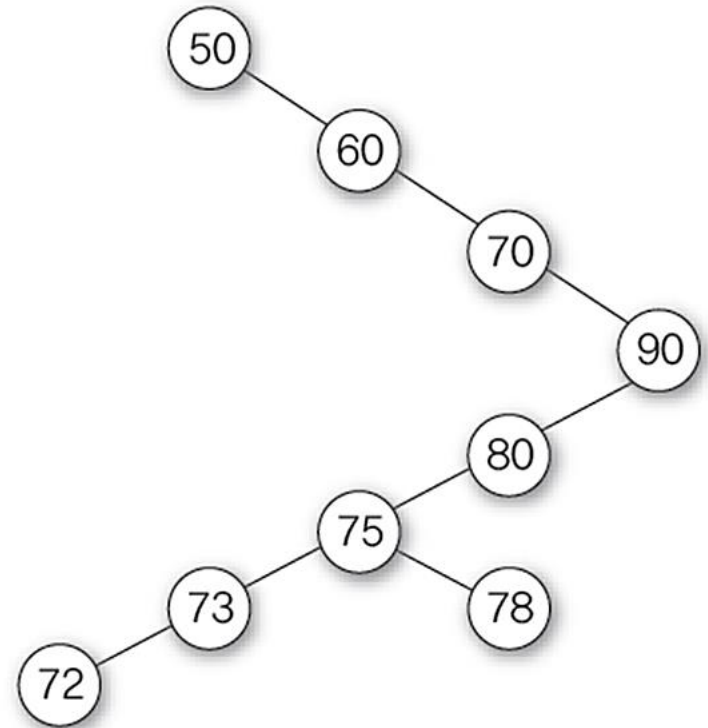
- ⑨ 노드 78 삽입 : 노드 75의 오른쪽 자식 노드로 노드 78을 삽입. 노드 80의 균형 인수가 +2가 되면서 LR 유형의 불균형이 발생. 1차 회전에서 RR 회전을 하고 2차 회전에서 LL 회전을 하는 LR 회전을 수행하여 균형을 회복



❖ 같은 원소를 같은 순서로 삽입할 경우의 AVL 트리와 이진 탐색 트리 비교



(a) AVL 트리



(b) 이진 탐색 트리

❖ AVL 트리 구현하기 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#define MAX(a,b) ((a>b)?a:b)

typedef int element;

typedef struct treeNode {
    element key;
    struct treeNode *left;
    struct treeNode *right;
}treeNode;
```

❖ AVL 트리 구현하기 프로그램

```
treeNode* LL_rotate(treeNode *parent)
{
    treeNode *child = parent->left;
    parent->left = child->right;
    child->right = parent;
    return child;
}

treeNode* RR_rotate(treeNode *parent)
{
    treeNode *child = parent->right;
    parent->right = child->left;
    child->left = parent;
    return child;
}
```

❖ AVL 트리 구현하기 프로그램

```
treeNode* LR_rotate(treeNode *parent)
{
    treeNode *child = parent->left;
    parent->left = RR_rotate(child);
    return LL_rotate(parent);
}

treeNode* RL_rotate(treeNode *parent)
{
    treeNode *child = parent->right;
    parent->right = LL_rotate(child);
    return RR_rotate(parent);
}
```

❖ AVL 트리 구현하기 프로그램

```
// 서브 트리의 높이를 구하는 연산
int getHeight(treeNode* p)
{
    int height = 0;
    if (p != NULL)
        height = MAX(getHeight(p->left), getHeight(p->right)) + 1;
    return height;
}

// 서브 트리의 높이를 이용해 균형 인수 BF를 구하는 연산
int getBF(treeNode* p)
{
    if (p == NULL)
        return 0;
    return getHeight(p->left) - getHeight(p->right);
}
```

❖ AVL 트리 구현하기 구현하기 프로그램

```
// BF를 검사하여 불균형이 발생한 경우, 회전 연산 호출
treeNode* rebalance(treeNode** p)
{
    int BF = getBF(*p);
    if (BF > 1) {
        if (getBF((*p)->left) > 0)
            *p = LL_rotate(*p);
        else
            *p = LR_rotate(*p);
    }
    else if (BF < -1) {
        if (getBF((*p)->right) < 0)
            *p = RR_rotate(*p);
        else
            *p = RL_rotate(*p);
    }
    return *p;
}
```


❖ AVL 트리 구현하기 구현하기 프로그램

```
// AVL 트리에 노드를 삽입하는 연산: 이진 탐색 연산처럼 삽입한 후에, rebalance() 호출
treeNode* insert_AVL_Node(treeNode** root, element x)
{
    if (*root == NULL) {
        *root = (treeNode*)malloc(sizeof(treeNode));
        (*root)->key = x;
        (*root)->left = NULL;
        (*root)->right = NULL;
    }
    else if (x < (*root)->key) {
        (*root)->left = insert_AVL_Node(&((*root)->left), x);
        *root = rebalance(root);
    }
}
```

❖ AVL 트리 구현하기 구현하기 프로그램

```
else if (x >(*root)->key) {
    (*root)->right = insert_AVL_Node(&((*root)->right), x);
    *root = rebalance(root);
}
else {
    printf("Wn 이미 같은 키가 있습니다! Wn");
    exit(1);
}
return *root;
}
```

❖ AVL 트리 구현하기 프로그램

```
// 이진 탐색 트리에 노드를 삽입하는 연산
treeNode* insert_BST_Node(treeNode *p, element x)
{
    treeNode *newNode;
    if (p == NULL) {
        newNode = (treeNode*)malloc(sizeof(treeNode));
        newNode->key = x;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    else if (x < p->key)
        p->left = insert_BST_Node(p->left, x);
    else if (x > p->key)
        p->right = insert_BST_Node(p->right, x);
    else
        printf("\n 이미 같은 키가 있습니다! \n");

    return p;
}
```

❖ AVL 트리 구현하기 프로그램

```
// 이진 탐색 트리와 AVL 트리에서 키값 x를 탐색하는 연산
treeNode* searchTree(treeNode* root, element x)
{
    treeNode* p;
    int count = 0;
    p = root;
    while (p != NULL) {
        count++;
        if (x < p->key)
            p = p->left;
        else if (x == p->key) {
            printf("%3d번째에 탐색 성공", count);
            return p;
        }
        else
            p = p->right;
    }
    count++;
    printf("%3d번째에 탐색 실패! 찾는 키가 없습니다!", count);
    return p;
}
```

❖ AVL 트리 구현하기 프로그램

```
// 이진 탐색 트리를 중위 순회하면서 출력하는 연산
void displayInorder(treeNode* root)
{
    if (root) {
        displayInorder(root->left);
        printf("_%d ", root->key);
        displayInorder(root->right);
    }
}
```

❖ AVL 트리 구현하기 프로그램

```
int main()
{
    treeNode* root_AVL = NULL;
    treeNode* root_BST = NULL;

    //////////////////////////////////////
    root_AVL = insert_AVL_Node(&root_AVL, 50);    // AVL 트리 만들기
    insert_AVL_Node(&root_AVL, 60);
    insert_AVL_Node(&root_AVL, 70);
    insert_AVL_Node(&root_AVL, 90);
    insert_AVL_Node(&root_AVL, 80);
    insert_AVL_Node(&root_AVL, 75);
    insert_AVL_Node(&root_AVL, 73);
    insert_AVL_Node(&root_AVL, 72);
    insert_AVL_Node(&root_AVL, 78);

    printf("\n ***** AVL 트리 출력 ***** \n\n");
    displayInorder(root_AVL);
}
```

❖ AVL 트리 구현하기 프로그램

```
printf("\n\n AVL 트리에서 70 탐색 : ");
searchTree(root_AVL, 70);
printf("\n\n AVL 트리에서 72 탐색 : ");
searchTree(root_AVL, 72);
printf("\n\n AVL 트리에서 76 탐색 : ");
searchTree(root_AVL, 76);

////////////////////////////////////
root_BST = insert_BST_Node(root_BST, 50); //BST 만들기
insert_BST_Node(root_BST, 60);
insert_BST_Node(root_BST, 70);
insert_BST_Node(root_BST, 90);
insert_BST_Node(root_BST, 80);
insert_BST_Node(root_BST, 75);
insert_BST_Node(root_BST, 73);
insert_BST_Node(root_BST, 72);
insert_BST_Node(root_BST, 78);

printf("\n\n\n ***** BST 출력 ***** \n\n");
displayInorder(root_BST);
```

❖ AVL 트리 구현하기 프로그램

```
printf("\n\n BST에서 70 탐색 : ");  
searchTree(root_BST, 70);  
  
printf("\n\n BST에서 72 탐색 : ");  
searchTree(root_BST, 72);  
  
printf("\n\n BST에서 76 탐색 : ");  
searchTree(root_BST, 76);  
  
getchar();  
}
```


❖ AVL 트리 구현하기 프로그램 : 교재 372p ~ 376p

■ 실행결과

```
C:\₩한국산업기술대₩2020년1학기₩자료구조와알고리즘₩연습문제₩제7...  -  □  X

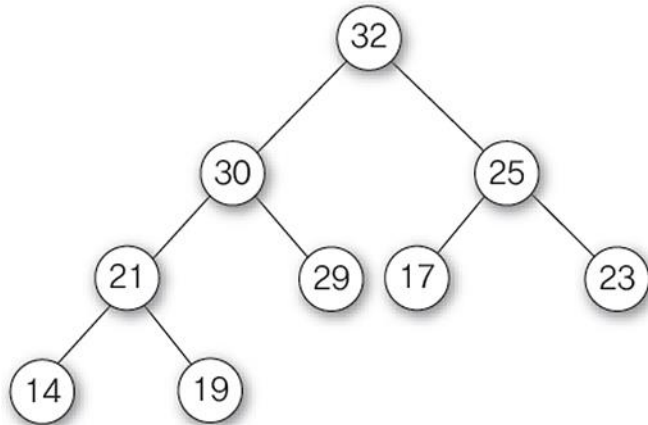
***** AVL 트리 출력 *****
_50 _60 _70 _72 _73 _75 _78 _80 _90
AVL 트리에서 70 탐색 :   1번째에 탐색 성공
AVL 트리에서 72 탐색 :   4번째에 탐색 성공
AVL 트리에서 76 탐색 :   5번째에 탐색 실패! 찾는 키가 없습니다!

***** BST 출력 *****
_50 _60 _70 _72 _73 _75 _78 _80 _90
BST에서 70 탐색 :   3번째에 탐색 성공
BST에서 72 탐색 :   8번째에 탐색 성공
BST에서 76 탐색 :   8번째에 탐색 실패! 찾는 키가 없습니다!
```

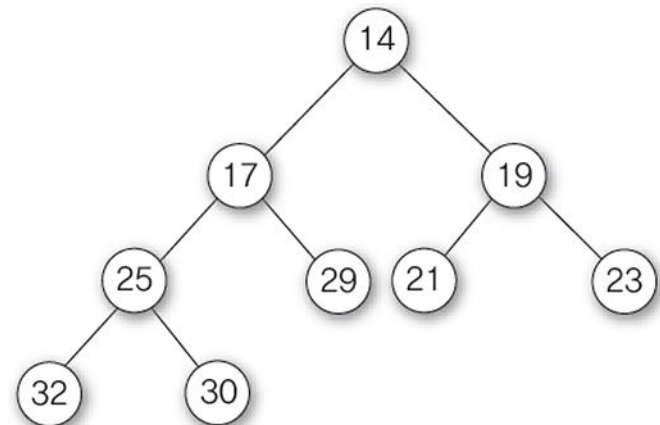
❖ 히프(heap)의 개념

- 완전 이진 트리에 있는 노드 중에서 키 값이 가장 큰 노드나 키 값이 가장 작은 노드를 찾기 위해서 만든 자료구조
- 최대 히프(max heap)
 - 키 값이 가장 큰 노드를 찾기 위한 완전 이진 트리
 - {부모 노드의 키값 $\geq$ 자식노드의 키값}
 - 루트 노드 : 키 값이 가장 큰 노드
- 최소 히프(min heap)
 - 키 값이 가장 작은 노드를 찾기 위한 완전 이진 트리
 - {부모 노드의 키값 $\leq$ 자식노드의 키값}
 - 루트 노드 : 키값이 가장 작은 노드

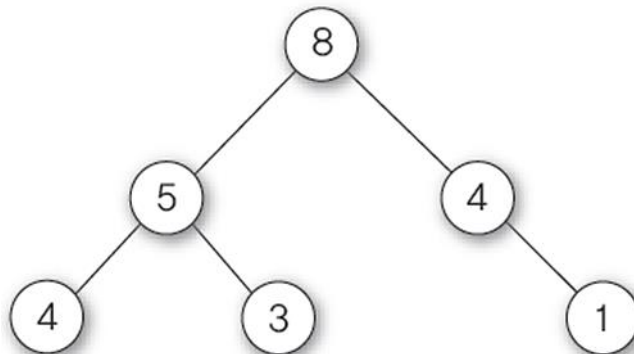
❖ 히프(heap)의 개념



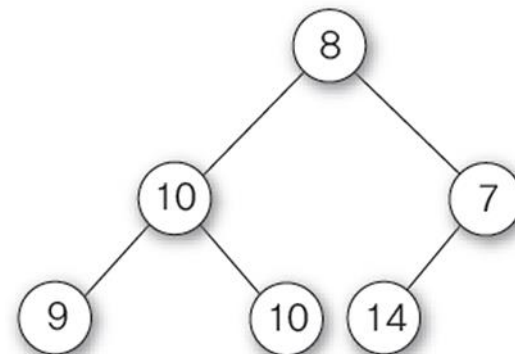
최대 히프



최소 히프



완전 이진 트리가 아님



노드 사이의 키 값 관계가 미성립

❖ 히프의 추상 자료형

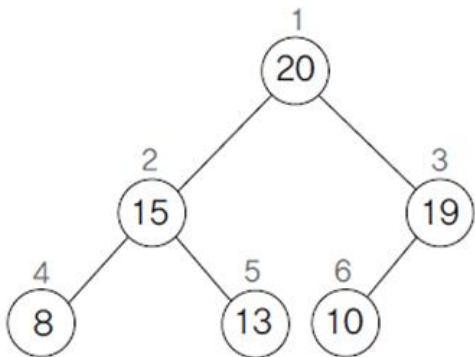
ADT	히프의 추상 자료형
ADT Heap	
	데이터 : n 개의 원소로 구성된 완전 이진 트리로서 각 노드의 키값은 그의 자식 노드의 키값보다 크거나 같다. (부모 노드의 키값 $\geq$ 자식 노드의 키값)
	연산 :
	heap $\in$ Heap; item $\in$ Element;
	// 공백 히프를 생성하는 연산
	createHeap() ::= create an empty heap;
	// 히프가 공백인지 검사하는 연산
	isEmpty(heap) ::= if (heap is empty) then return true; else return false;
	// 히프의 적당한 위치에 원소(item)를 삽입하는 연산
	insertHeap(heap, item) ::= insert item into heap;
	// 히프의 키값이 가장 큰 원소를 삭제하고 반환하는 연산
	deleteHeap(heap) ::= if (isEmpty(heap)) then return error; else { item $\leftarrow$ 히프에서 가장 큰 원소; remove {히프에서 가장 큰 원소}; return item; }
	End Heap()

❖ 히프의 삽입 연산

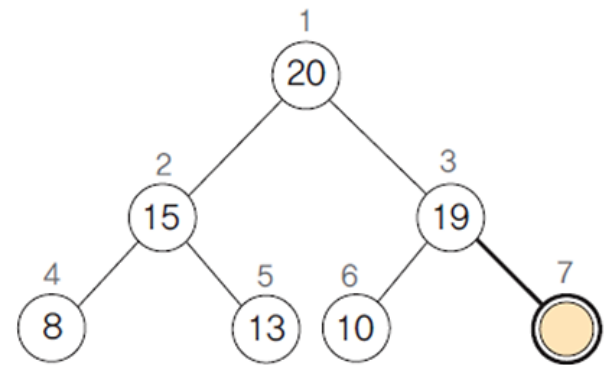
- 1단계 : 완전 이진 트리의 조건이 만족하도록 다음 자리를 확장
 - 노드가 n 개인 완전 이진 트리에서 다음 노드 확장 자리는 $n+1$ 번의 노드
 - $n+1$ 번 자리에 노드를 확장하고, 그 자리에 삽입할 원소를 임시 저장
- 2단계 : 부모 노드와 크기 조건이 만족하도록 삽입 원소의 위치를 찾음
 - 현재 위치에서 부모 노드와 비교하여 크기 관계를 확인
 - {현재 부모노드의 키값 $\geq$ 삽입 원소의 키값}의 관계가 성립하지 않으면, 부모 노드의 원소와 삽입 원소의 자리를 서로 바꿈

❖ 히프에서의 삽입 연산 : 17을 삽입하는 경우

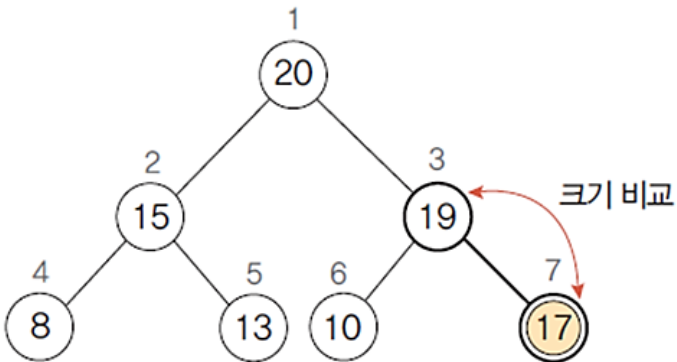
- 노드를 확장하여 임시로 저장한 위치에서 부모노드와 크기를 비교하여 히프의 크기 관계가 성립하므로, 현재 위치를 삽입 원소의 자리로 확정



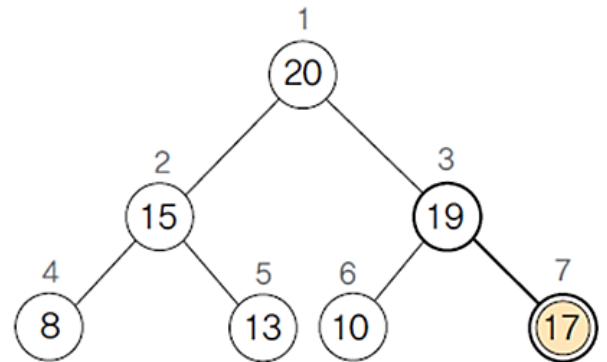
삽입 전의 히프



완전 이진 트리의 다음 자리인 7번 노드 확장

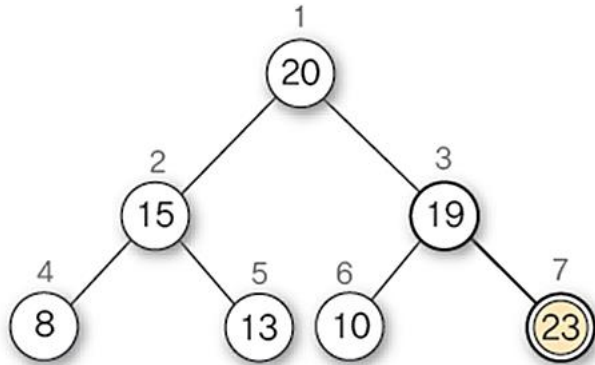


삽입 원소 17의 자리 확정

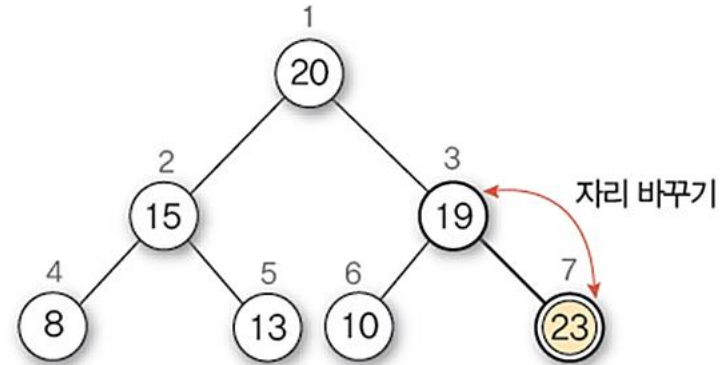


❖ 히프에서 삽입 연산 : 23을 삽입하는 경우

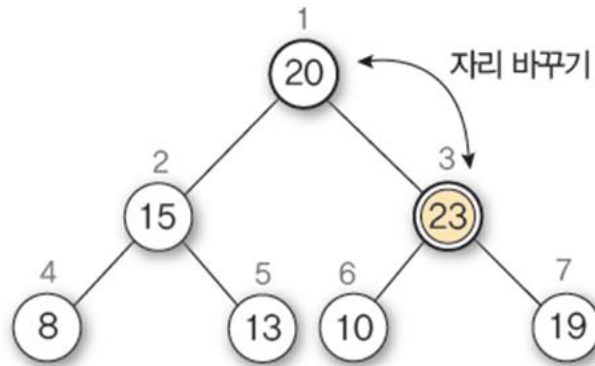
① 확장한 자리에 삽입원소 23을 임시저장



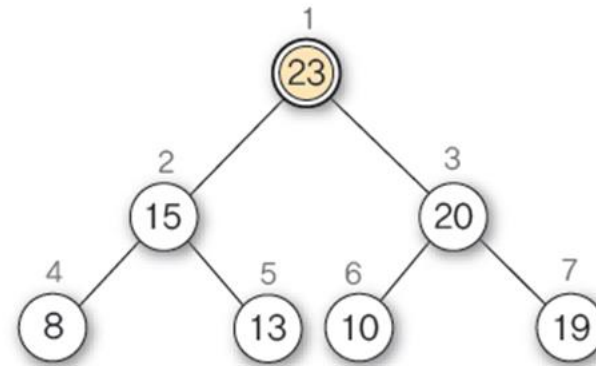
② (부모 노드 19 < 삽입노드 23)이므로 자리 바꾸기



③ (부모 노드 20 < 삽입노드 23)이므로 자리 바꾸기



④ 더 이상 비교할 부모 노드가 없으므로 자리 확정



❖ 히프에서의 삽입 연산

알고리즘

최대 히프의 노드 삽입

```
insertHeap(heap, item)
    if (n = heapSize) then heapFull();
    ①  $n \leftarrow n+1$ ;
    ② for ( $i \leftarrow n$ ; ;) do {
        if ( $i = 1$ ) then exit;
        ③ if ( $\text{item} \leq \text{heap}[i/2]$ ) then exit;
        ④  $\text{heap}[i] \leftarrow \text{heap}[i/2]$ ;
        ⑤  $i \leftarrow i/2$ 
    }
    ⑥  $\text{heap}[i] \leftarrow \text{item}$ ;
end insertHeap()
```


❖ 히프에서의 삽입 연산

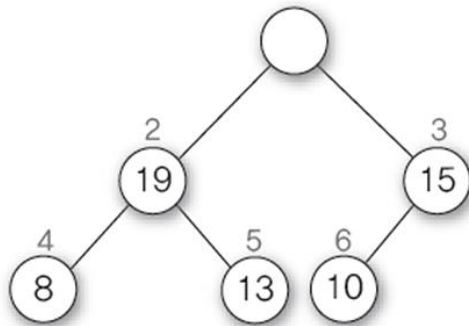
- ① 현재 히프의 크기를 하나 증가시켜서 노드 위치를 확장 한다.
- ② 확장한 노드 번호가 현재의 삽입 위치 i 가 된다.
- ③ 삽입할 원소 $item$ 과 부모 노드 $heap[i/2]$ 를 비교하여 $item$ 이 부모 노드 보다 작거나 같으면 현재의 삽입 위치 i 를 삽입 원소의 위치로 확정하고 ⑥을 수행한다.
- ④ 삽입할 원소 $item$ 이 부모 노드 보다 크면, 부모 노드와 자식 노드의 자리를 바꾸어 최대 히프의 관계를 만들어야 하므로 부모 노드 $heap[i/2]$ 를 현재의 삽입 위치 $heap[i]$ 에 저장한다.
- ⑤ $i/2$ 를 삽입 위치 i 로 하여, ②~⑤를 반복하면서 $item$ 을 삽입할 위치를 찾는다.
- ⑥ 찾은 위치에 삽입할 노드 $item$ 을 저장하면, 최대 히프의 재구성 작업이 완성되므로 삽입 연산을 종료한다.

❖ 히프의 삭제 연산

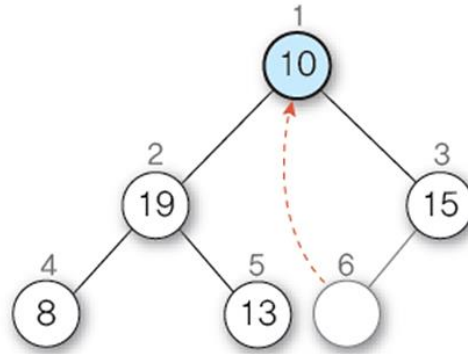
- 히프에서 루트 노드의 원소만을 삭제 할 수 있음
- **1단계** : 노드의 원소를 삭제하여 반환
- **2단계** : 원소의 개수가 $n-1$ 개로 줄었으므로, 노드의 수가 $n-1$ 인 완전 이진 트리로 조정
 - 노드가 n 개인 완전 이진 트리에서 노드 수 $n-1$ 개의 완전 이진 트리가 되기 위해서 마지막 노드, 즉 n 번 노드를 삭제
 - 삭제된 n 번 노드에 있던 원소는 비어있는 루트노드에 임시 저장
- **3단계** : 완전 이진 트리 내에서 루트에 임시 저장된 원소의 제 자리를 찾음
 - 현재 위치에서 자식노드와 비교하여 크기 관계를 확인
 - {임시 저장 원소의 키값 $\geq$ 현재 자식노드의 키값}의 관계가 성립하지 않으면, 현재 자식노드의 원소와 임시 저장 원소의 자리를 서로 바꿈

❖ 히프의 삭제 연산

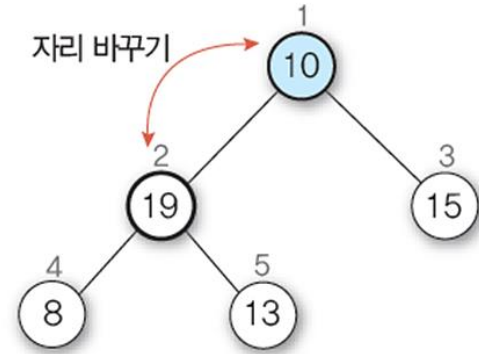
① 루트 노드의 원소 삭제



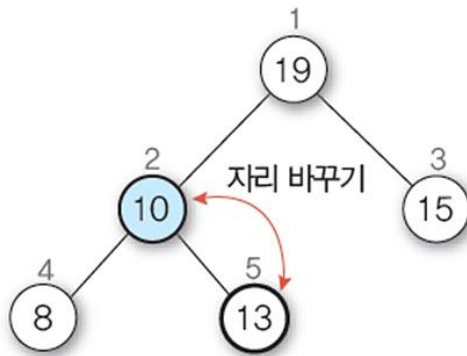
② 마지막 노드 삭제 후 원소를 루트로 이동



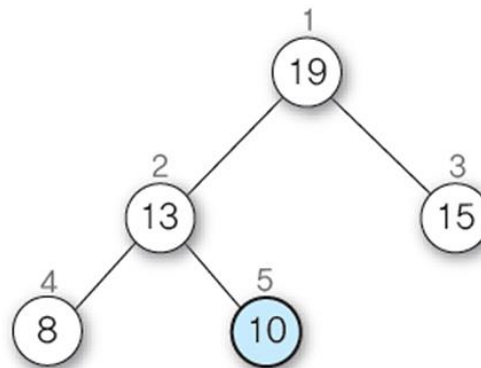
③ (삽입 노드 10 < 자식 노드 19)이므로 자리 바꾸기



④ (삽입 노드 10 < 자식 노드 13)이므로 자리 바꾸기



⑤ 자리 확정



❖ 히프의 삭제 연산

알고리즘

최대 히프의 노드 삭제

```
deleteHeap(heap)
  if (n = 0) then return error;
  ① item ← heap[1];
  ② temp ← heap[n];
  ③ n ← n-1;
  ④ i ← 1;
    j ← 2;
  ⑤ { while (j ≤ n) do {
        if (j < n) then
          if (heap[j] < heap[j+1]) then j ← j+1;
        if (temp ≥ heap[j]) then exit;
        heap[i] ← heap[j];
  ⑥ {   i ← j;
        j ← j*2;
      }
  ⑦ heap[i] ← temp;
  ⑧ return item;
end deleteHeap( )
```

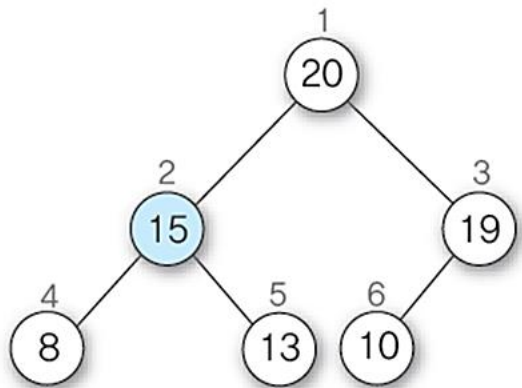
❖ 히프의 삭제 연산

- ① 루트 노드 $\text{heap}[1]$ 을 item 에 저장
- ② 마지막 노드의 원소 $\text{heap}[n]$ 을 temp 에 임시로 저장
- ③ 마지막 노드를 삭제하였으므로 히프 배열의 원소 개수가 하나 감소
- ④ 마지막 노드의 원소였던 temp 의 임시 저장위치 i 는 루트 노드 자리인 1번이 된다.
- ⑤ 현재 저장위치에서 자식 노드 $\text{heap}[j]$ 와 $\text{heap}[j+1]$ 이 있을 때, 둘 중에서 키 값이 큰 자식 노드의 키 값과 temp 를 비교하여, temp 가 크거나 같으면 현재의 임시 저장 위치를 temp 자리로 확정하고 ⑦을 수행
- ⑥ temp 가 자식 노드 $\text{heap}[j]$ 보다 작으면 자식 노드와 자리를 바꾸고 ⑤~⑥을 반복하면서 temp 의 자리를 찾는다.
- ⑦ 찾은 위치에 temp 를 저장하여 최대 히프의 재구성 작업을 완성
- ⑧ 처음에 삭제된 루트 노드를 저장한 item 을 반환하는 것으로 삭제 연산을 종료한다.

히프의 개념과 연산 및 구현

❖ 순차 자료 구조를 이용한 히프의 구현

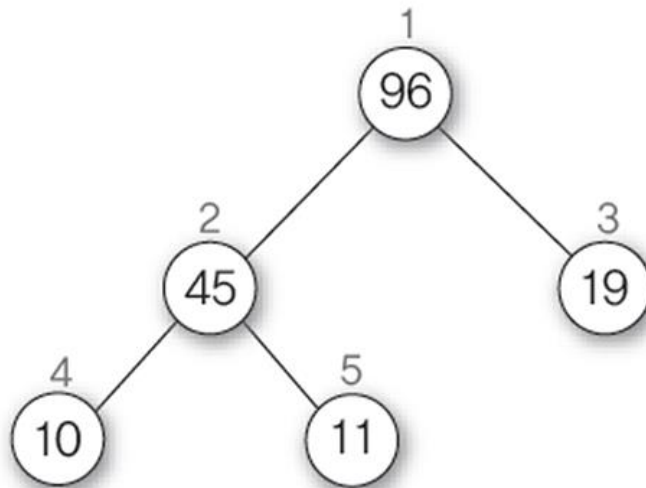
- 1차원 배열의 순차 자료 구조를 이용하면 인덱스 관계를 이용하여 부모 노드를 찾기 쉬움
- 1차원 배열을 이용한 히프의 표현 예



[0]		
[1]	20	부모 노드의 인덱스 = $\lfloor i/2 \rfloor = 2/2 = 1$
[2]	15	왼쪽 자식 노드의 인덱스 = $i \times 2 = 2 \times 2 = 4$
[3]	19	
[4]	8	
[5]	13	오른쪽 자식 노드의 인덱스 = $i \times 2 + 1 = 2 \times 2 + 1 = 5$
[6]	10	

❖ 순차 자료구조를 이용한 히프 프로그램

- 공백 히프에 원소 다섯 개(10, 45, 19, 11, 96)를 삽입하여 최대 히프를 구성하고, 삭제 연산을 수행하여 삭제된 원소를 출력
 - 최대 히프의 루트는 히프에서 가장 큰 노드가 되므로, 원소 개수 만큼 삭제 연산을 수행하여 출력하면 큰 값부터 작은 값의 내림차순으로 출력되는 것을 확인



❖ 순차 자료구조를 이용한 최대 히프 구현 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#define MAX_ELEMENT 100

// 히프에 대한 1차원 배열과 히프 원소의 개수를 구조체로 묶어서 선언
typedef struct {
    int heap[MAX_ELEMENT];
    int heap_size;
} heapType;

// 공백 히프를 생성하는 연산
heapType* createHeap()
{
    heapType *h = (heapType*)malloc(sizeof(heapType));
    h->heap_size = 0;
    return h;
}
```


❖ 순차 자료구조를 이용한 최대 히프 구현 프로그램

```
// 히프에 item을 삽입하는 연산
void insertHeap(heapType *h, int item)
{
    int i;
    h->heap_size = h->heap_size + 1;
    i = h->heap_size;
    while ((i != 1) && (item > h->heap[i / 2]))
    {
        h->heap[i] = h->heap[i / 2];
        i /= 2;
    }
    h->heap[i] = item;
}
```

❖ 순차 자료구조를 이용한 최대 히프 구현 프로그램

```
// 히프의 루트를 삭제하여 반환하는 연산
int deleteHeap(heapType *h)
{
    int parent, child;
    int item, temp;
    item = h->heap[1];
    temp = h->heap[h->heap_size];
    h->heap_size = h->heap_size - 1;
    parent = 1;
    child = 2;
    while (child <= h->heap_size) {
        if ((child < h->heap_size) && (h->heap[child]) < h->heap[child + 1])
            child++;
        if (temp >= h->heap[child]) break;
        else {
            h->heap[parent] = h->heap[child];
            parent = child;
            child = child * 2;
        }
    }
    h->heap[parent] = temp;
    return item;
}
```

❖ 순차 자료구조를 이용한 최대 히프 구현 프로그램

```
// 1차원 배열 히프의 내용을 출력하는 연산
void printHeap(heapType *h)
{
    int i;
    printf("Heap : ");
    for (i = 1; i <= h->heap_size; i++)
        printf("[%d] ", h->heap[i]);
}
```

❖ 순차 자료구조를 이용한 최대 히프 구현 프로그램

```
void main()
{
    int i, n, item;
    heapType *heap = createHeap();
    insertHeap(heap, 10);
    insertHeap(heap, 45);
    insertHeap(heap, 19);
    insertHeap(heap, 11);
    insertHeap(heap, 96);

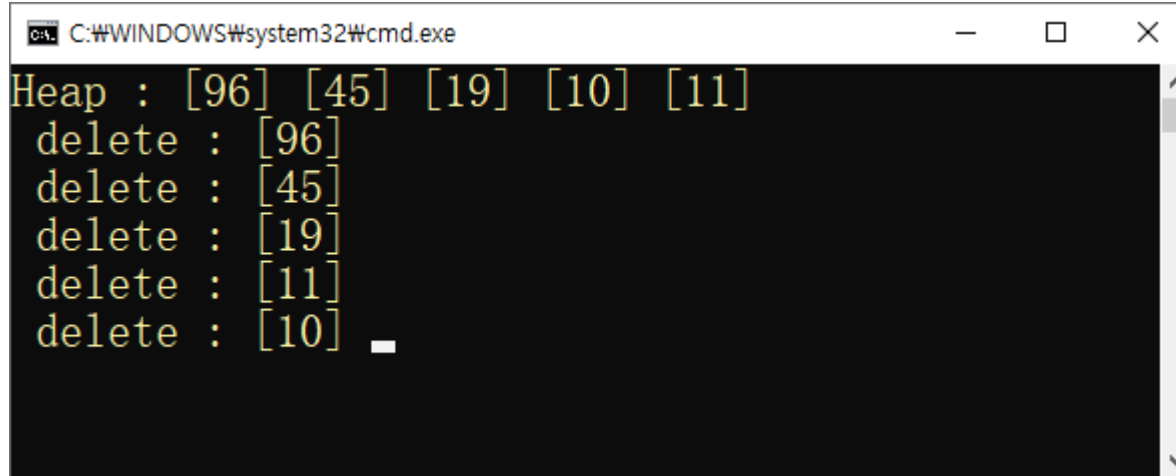
    printHeap(heap);

    n = heap->heap_size;
    for (i = 1; i <= n; i++) {
        item = deleteHeap(heap);
        printf("\n delete : [%d] ", item);
    }
    getchar();
}
```

힉프의 개념과 연산 및 구현

❖ 순차 자료구조를 이용한 최대 힉프 구현 프로그램 : [교재 384p](#)

■ 실행 결과



```
C:\WINDOWS\system32\cmd.exe
Heap : [96] [45] [19] [10] [11]
delete : [96]
delete : [45]
delete : [19]
delete : [11]
delete : [10] _
```